Crypto Wallet Frontend Criteria

Техническое задание для frontend-части. React + Vite + JavaScript

Цель документа - зафиксировать, какой frontend нужно построить для учебного проекта Crypto Wallet: структуру проекта, технологический стек, страницы, UI, API-интеграцию, auth-flow, стилизацию и критерии готовности.

Проект небольшой. Не нужно переусложнять архитектуру, ставить тяжелые UI-kit, Redux, TypeScript или сложный form/state/data-fetching слой. Основная цель - быстро получить рабочий интерфейс к уже готовому backend API.

1. Краткое решение

- Frontend делаем на **React**, потому что основной рабочий стек пользователя уже связан с React.
- Сборщик - **Vite**. Webpack не нужен.
- Язык - **JavaScript**, без TypeScript.
- Стили - **CSS Modules** для страниц и компонентов + обычный global CSS для reset, typography и CSS variables.
- Маршрутизация - **react-router-dom**.
- Запросы - **native fetch** через собственный `apiClient`. Axios не нужен.
- Глобальное состояние - **React Context / useState** только для auth-state. Redux не нужен.
- Access token хранится в памяти приложения, refresh token остается в httpOnly cookie.

2. Расположение проекта

Корень `crypto-wallet` остается backend-проектом. Frontend создается отдельным Vite-проектом в папке `frontend` на одном уровне с backend `src`.

```
crypto-wallet/
  package.json          # backend package.json
  .env                  # backend env
  src/                  # backend source
    app.js
    server.js
    modules/
    plugins/
    db/

  frontend/             # frontend app
    package.json        # frontend package.json
    index.html
    src/
      main.jsx
      App.jsx
```

Локальный запуск:

Часть	Где запускать	Команда	URL
Backend	из корня `crypto-wallet`	`npm run dev`	`http://localhost:3000`
Frontend	из `crypto-wallet/frontend`	`npm run dev`	`http://localhost:5173`

Локально frontend и backend имеют разные origins, поэтому backend должен разрешать CORS для `http://localhost:5173` с `credentials: true`. В production можно будет использовать `VITE_API_URL=/api` и проксирование через Nginx.

3. Утвержденный технологический стек

Зона	Решение	Причина
Framework	React	Пользователь уже знает React. Проект небольшой, нет смысла переключаться на Vue.
Build tool	Vite	Быстрый старт, простой dev-server, минимум конфигурации.
Language	JavaScript	TypeScript пока не нужен. Важнее собрать понятный рабочий UI.
Routing	react-router-dom	Нужны отдельные routes: `/login`, `/register`, `/dashboard`.
Styling	CSS Modules + CSS variables	Локальные классы без конфликтов, но без тяжелых styling-библиотек.
API layer	native fetch + custom apiClient	Достаточно для небольшого количества endpoints. Axios не нужен.
State	useState, useEffect, Context	Достаточно для auth, user, wallet, transactions, loading/error.

Что не используем

- Vue - финальное решение в пользу React.
- TypeScript - не используем на первом этапе.
- Redux - избыточен для текущего объема состояния.
- TanStack Query - можно добавить позже, сейчас native fetch понятнее.
- Axios - не нужен, fetch достаточно.
- React Hook Form - формы маленькие, controlled inputs через useState достаточно.
- Tailwind, MUI, styled-components - не используем, чтобы не раздувать проект.
- Webpack - не нужен, Vite закрывает потребности.

4. UI scope и страницы

Frontend должен закрывать базовый пользовательский сценарий: регистрация, вход, автоматическое восстановление сессии через refresh cookie, просмотр кошелька, просмотр транзакций, создание deposit/withdrawal, logout.

Route	Назначение	Основные элементы
`/login`	Вход в аккаунт	Email, password, submit, ссылка на регистрацию, error message.
`/register`	Создание аккаунта	Email, password, submit, ссылка на login, error message.
`/dashboard`	Главный экран кошелька	Header, user email, logout, balances, transaction form, transactions table.
*	Fallback route	Redirect на `/dashboard` если user авторизован, иначе на `/login`.

Auth pages

- Login и Register можно сделать отдельными страницами или общей Auth layout-страницей с разными route.
- Форма должна быть простой: email + password.
- При успешном login/register сохраняем accessToken и user в auth-state, затем переходим на `/dashboard`.
- Refresh token frontend не читает. Он хранится как httpOnly cookie, установленная backend.

Dashboard page

- Одна основная страница dashboard без вложенной сложной навигации.
- Верхняя панель: название Crypto Wallet, email пользователя, кнопка Logout.
- Основной layout: карточка wallet/balances, форма создания transaction, список transactions.
- На desktop можно использовать 2-3 колонки. На mobile колонки должны складываться вертикально.

5. Визуальный стиль

Стиль: чистый светлый crypto-dashboard. Можно вдохновляться общей аккуратностью Trust Wallet, но не копировать бренд, логотипы, иконки или точный интерфейс.

Token	Value	Use
`--color-bg`	`#F7FAFB`	Фон приложения
`--color-surface`	`#FFFFFF`	Карточки, формы, таблицы
`--color-text`	`#101828`	Основной текст
`--color-muted`	`#667085`	Вторичный текст
`--color-border`	`#E4E7EC`	Бордюры и разделители
`--color-primary`	`#00A6B4`	Основной бирюзово-голубой акцент
`--color-primary-dark`	`#00818C`	Hover/active state
`--color-danger`	`#D92D20`	Ошибки и опасные действия
`--color-success`	`#039855`	Успех, deposit indication

UI principles:

- Светлая тема на первом этапе. Dark theme не делать.
- Белый, серый, черный как база. Бирюзовый как основной акцент.
- Не делать визуально перегруженный trading-terminal.
- Использовать карточки, мягкие border-radius, аккуратные shadows.
- Кнопки и формы должны выглядеть цельно и одинаково.
- Не использовать внешние UI-kit. Все компоненты простые, самописные.

6. Backend API contract

Frontend должен интегрироваться с уже существующим Fastify backend. Base URL локально задается через `VITE_API_URL=http://localhost:3000`.

Auth endpoints:

- `POST /auth/register`
  Body: `{ email, password }`
  Auth: no.
  Result: создает user + wallet, возвращает user, wallet, accessToken, ставит refresh cookie.
- `POST /auth/login`
  Body: `{ email, password }`
  Auth: no.
  Result: возвращает user, accessToken, ставит refresh cookie.
- `POST /auth/refresh`
  Body: none.
  Auth: refresh cookie, `credentials: 'include'`.
  Result: если refresh cookie валидна, возвращает user и новый accessToken.
- `POST /auth/logout`
  Body: none.
  Auth: refresh cookie, `credentials: 'include'`.
  Result: очищает refresh cookie, возвращает success.

Protected endpoints:

- `GET /wallet`
  Auth: `Authorization: Bearer <accessToken>`.
  Result: возвращает wallet с balances.
- `GET /transactions`
  Auth: `Authorization: Bearer <accessToken>`.
  Result: возвращает список transactions текущего user.
- `GET /transactions/:id`
  Auth: `Authorization: Bearer <accessToken>`.
  Result: возвращает одну transaction текущего user.
- `POST /transactions`
  Auth: `Authorization: Bearer <accessToken>`.
  Body: `{ type, asset, amount }`.
  Result: создает deposit/withdrawal и обновляет balance.

Transaction body:

```
{
  "type": "deposit" | "withdrawal",
  "asset": "BTC" | "ETH" | "USDT",
  "amount": 1
}
```

Все успешные ответы backend возвращает в обертке `data`. Frontend должен читать `result.data`, а не весь response как бизнес-объект.

```
{
  "data": {
    "...": "..."
  }
}
```

7. Auth-flow на frontend

Access token хранится только в памяти React-приложения: Context/useState. Не сохранять accessToken в localStorage на первом этапе. Refresh token frontend не читает, потому что он находится в httpOnly cookie.

App bootstrap

- При старте приложения вызвать `POST /auth/refresh` с `credentials: 'include'`.
- Если refresh успешен - сохранить user и accessToken в auth-state, пустить на `/dashboard`.
- Если refresh неуспешен - очистить auth-state и показать `/login`.
- Пока идет refresh - показать loading screen, чтобы не мигал login/dashboard.

Login/register

- После успешного login/register сохранить accessToken и user в auth-state.
- Перенаправить пользователя на `/dashboard`.
- Refresh cookie backend установит сам через `Set-Cookie`.

Protected requests

```
Authorization: Bearer <accessToken>
```

Refresh/logout requests

```
fetch(url, {
  method: 'POST',
  credentials: 'include'
})
```

401 behavior

- Если protected request вернул 401, можно один раз попробовать `/auth/refresh`.
- Если refresh успешен - повторить исходный запрос с новым accessToken.
- Если refresh неуспешен - очистить auth-state и отправить на `/login`.
- Не делать бесконечный refresh loop.

8. Frontend folder structure

Структура должна быть простой и читаемой. Компоненты небольшие. API-запросы вынести отдельно от UI.

```
frontend/
  package.json
  index.html
  .env

  src/
    api/
      client.js
      auth.api.js
      wallet.api.js
      transactions.api.js

    components/
```

```
    Button/
      Button.jsx
      Button.module.css
    Card/
      Card.jsx
      Card.module.css
    Header/
      Header.jsx
      Header.module.css
    BalanceList/
      BalanceList.jsx
      BalanceList.module.css
    TransactionForm/
      TransactionForm.jsx
      TransactionForm.module.css
    TransactionList/
      TransactionList.jsx
      TransactionList.module.css

  pages/
    LoginPage/
      LoginPage.jsx
      LoginPage.module.css
    RegisterPage/
      RegisterPage.jsx
      RegisterPage.module.css
    DashboardPage/
      DashboardPage.jsx
      DashboardPage.module.css

  context/
    AuthContext.jsx

  styles/
    tokens.css
    globals.css

  App.jsx
  main.jsx
```

API layer responsibility:

- `client.js` хранит baseUrl, JSON parsing, credentials mode where needed, error handling.
- `auth.api.js` содержит login, register, refresh, logout.
- `wallet.api.js` содержит getWallet.
- `transactions.api.js` содержит getTransactions, getTransactionById, createTransaction.
- UI-компоненты не должны собирать URL руками в каждом месте.

9. Components and behavior

Component	Responsibility
Button	Единый внешний вид кнопок: primary, secondary, danger, disabled/loading.
Card	Обертка для dashboard-блоков: title, optional subtitle, content.
Header	App name, user email, Logout button.

Component	Responsibility
BalanceList	Показывает BTC, ETH, USDT balances. Обрабатывает empty/loading/error.
TransactionForm	Создает deposit/withdrawal. Поля: type, asset, amount.
TransactionList	Таблица или список transactions: id, type, asset, amount, date.
AuthContext	Хранит user, accessToken, isAuthenticated, isLoading, login, register, logout, refresh.

Dashboard data loading:

- После входа загрузить wallet и transactions.
- После успешного deposit/withdrawal перезагрузить wallet и transactions, чтобы UI отражал backend-state.
- Показывать loading state отдельно для dashboard initial load и для submit transaction.
- Ошибки backend показывать пользователю в понятном виде, без stack trace.

10. Styling details

Используем гибрид: глобальные CSS-файлы только для базовой настройки, а все конкретные стили страниц и компонентов - через CSS Modules.

`tokens.css`:

```css
:root {
  --color-bg: #f7fafb;
  --color-surface: #ffffff;
  --color-text: #101828;
  --color-muted: #667085;
  --color-border: #e4e7ec;

  --color-primary: #00a6b4;
  --color-primary-dark: #00818c;
  --color-danger: #d92d20;
  --color-success: #039855;

  --radius-sm: 8px;
  --radius-md: 12px;
  --radius-lg: 20px;

  --shadow-card: 0 12px 30px rgba(16, 24, 40, 0.08);
}
```

CSS Modules example:

```css
.card {
  background: var(--color-surface);
  border: 1px solid var(--color-border);
  border-radius: var(--radius-lg);
  box-shadow: var(--shadow-card);
}
```

Naming approach:

- В `module.css` можно использовать короткие имена: `.card`, `.title`, `.button`, `.field`, `.error`.
- Не нужен тяжелый BEM, потому что классы локальны для компонента.
- Не писать большие общие CSS-файлы со стилями всех компонентов.

11. Validation, loading, errors

Area	Requirement
Login/Register	Проверять, что email и password заполнены. Минимально показывать ошибку если backend вернул error.
Transaction amount	Amount должен быть числом больше 0. Пустые, отрицательные и нулевые значения не отправлять.
Transaction type	Select: deposit или withdrawal.
Asset	Select: BTC, ETH, USDT.
Loading	Disable submit button during request, показывать loading label/spinner text.
Errors	Показывать короткое сообщение рядом с формой или в верхней части карточки.
Empty states	Если transactions пустые, показать нормальный empty state, а не пустую таблицу.

Не показывать пользователю технические stack traces, raw Fastify logs или внутренние details, если они не нужны для UI.

12. Environment and local backend requirements

Frontend `.env`:

```
VITE_API_URL=http://localhost:3000
```

В production можно заменить на:

```
VITE_API_URL=/api
```

Backend CORS для локальной разработки:

```
origin: ['http://localhost:5173']
credentials: true
```

Credentials нужны для refresh/logout, потому что refresh token хранится в httpOnly cookie.

13. Acceptance criteria

Frontend считается готовым на первом этапе, если выполнены эти критерии.

- Проект находится в `crypto-wallet/frontend` и имеет свой `package.json`.
- `npm run dev` запускает frontend на Vite.
- Есть routes `/login`, `/register`, `/dashboard`.
- Register создает пользователя, получает accessToken и переводит на dashboard.
- Login получает accessToken и переводит на dashboard.
- При reload приложения вызывается `/auth/refresh` с `credentials: include`.
- Если refresh успешен, пользователь остается авторизован.
- Если refresh неуспешен, пользователь попадает на login.
- Dashboard показывает user email, wallet balances и transaction list.
- Transaction form позволяет создать deposit и withdrawal.
- После transaction UI обновляет balances и transactions.
- Logout вызывает `/auth/logout`, очищает auth-state и отправляет на login.
- Protected API-запросы отправляют `Authorization: Bearer <accessToken>`.
- Refresh/logout запросы отправляют `credentials: include`.

- Ошибки и loading states отображаются в UI.
- Стили реализованы через CSS Modules + global `tokens.css`/`globals.css`.
- Проект не использует TypeScript, Redux, Tailwind, MUI, Axios, React Hook Form или TanStack Query на первом этапе.

14. Final instruction for AI frontend generator

Сгенерировать небольшой frontend для Crypto Wallet на React + Vite + JavaScript. Не усложнять архитектуру. Соблюдать структуру папок, использовать CSS Modules, react-router-dom, native fetch через apiClient, AuthContext для auth-state. Реализовать login, register, refresh on app start, logout, dashboard с balances, transactions и формой deposit/withdrawal. Визуальный стиль - светлый, аккуратный, бело-серо-черный с насыщенным бирюзово-голубым акцентом `#00A6B4`.